Progress counts, self-imitation and hand-written skills on Craftax at 1M and 1B steps

Paweł Pozorski[1]

[1] Warsaw University of Technology, Poland

pozorski.paul@gmail.com

Abstract

We compare three baselines on Craftax-Symbolic (PPO, PPO-RNN, PQN-RNN) with four methods built on them: a count-based progress bonus, self-imitation, Q-learning over hand-written skills and Dyna-Q over a skill-level model, plus a combination of the two best. Every agent trains for 1M and 1B steps under one protocol, with 10 and 3 seeds. At 1M the progress bonus on PPO-RNN scores 2.27 % against 2.19 % for PPO-RNN, a difference inside the seed noise, and the skill methods are far behind. At 1B the order of the baselines changes: PQN-RNN rises from the weakest to the strongest control, no method resolvably beats its control, and the rank correlation between the two budgets is only 0.67. A rerun of the baselines on the cluster software stack shows no stack effect. We release code, results and a demo.

Index Terms: reinforcement learning, Craftax, exploration, self-imitation, skills, benchmark

1. Introduction

Craftax [1] is an open-ended survival benchmark in JAX: 67 achievements across nine floors, a maximum reward of 226, and a scoreboard at one million (1M) and one billion (1B) environment steps. Its published baselines are PPO variants and a recurrent PQN. We ask which cheap additions to such baselines help, and whether a result at 1M still holds at 1B. Contributions:

- a controlled comparison of three baselines, four methods and their combination under one protocol, at both budgets;
- a 1M-versus-1B analysis (rank agreement, advantage over the base control, learning speed, precision at three seeds) and a check that the software stack does not explain it;
- decision-tree interpretations of every final policy;
- an open release of code, results and an interactive demo.

2. Related work

The Craftax paper reports PPO, a recurrent PPO and three intrinsic-reward variants (RND, ICM, E3B) [1]; PQN with a recurrent network reaches the best published 1B score among the baselines we reproduce [2]. We compare only entries of the same protocol (training score, same budget). Self-imitation [3] and Dyna-style planning [4] are classic ideas we adapt to Craftax's achievement structure. For statistics we follow [5]: interquartile mean, bootstrap intervals and probability of improvement.

3. Methods

Controls. PPO [6] with an MLP; PPO-RNN, the same with a GRU memory; PQN-RNN, online Q(λ)-learning on the batch of many parallel environments, without replay buffer or target network. Every method is compared with its base control, and only within its block.

Count-based progress bonus. An intrinsic reward $\beta_t/\sqrt{N[\phi(s_{t+1})]}$, where ϕ summarises game progress (floor, tools, armour, spells, bucketed inventory) and N counts over the whole run; variants on PPO-RNN and PQN-RNN, and a SimHash-of-the-observation ablation.

Self-imitation (SIL). PPO-RNN plus a buffer of the steps that led to each achievement and an extra loss that imitates a stored action only where the return that followed beat the critic [3].

Skill methods (use hand-written skills). SMDP Q-learning: a table over hashed progress keys and 25 hand-written options learns on the native reward, one shared recurrent Q-network learns the skills on success pseudo-rewards. Dyna-Q adds a learned lookup model of skill outcomes and planning updates on it. They are reported in their own block, because the skills are given.

Combination. Progress bonus and SIL on PPO-RNN, the best two compatible methods by the rank rule of both budgets (the re-check at 1B kept the pair).

4. Experimental setup

Benchmark and budgets. Craftax-Symbolic at 1M and 1B steps.

Seeds. 3 per configuration while tuning, 10 per final at 1M, 3 per final at 1B.

Tuning. Two stages at 1M (coarse, then fine grid); the 1B runs reuse the 1M knobs and the published 1B settings of the shared PPO part, so a method can look worse at 1B than its best setting.

Score. The mean return of the training episodes over the last 2 % of the budget (at least 100 episodes) as a share of the 226 points; a separate evaluation on 64 fixed seeds, stochastic and greedy, is reported alongside.

Statistics. Mean $\pm$ s.e., interquartile mean (IQM) with 95 % bootstrap intervals over seeds, probability of improvement (PoI) and Holm-corrected Mann–Whitney tests at 1M; 1B has three seeds, so we give intervals and no tests.

Hardware and software. 1M on a laptop (Apple silicon), 1B on an A100/H100 cluster; JAX 0.11.2, craftax 1.6.1 (1M) and 1.7.0 (1B), MLflow for every metric. Because budget and stack differ, we reran the three controls at 1M on the cluster stack (10 seeds each).

Table 1: *Final training score (% of the maximum 226) at 1M (10 seeds) and 1B (3 seeds, indicative). Mean ± s.e.; IQM with a 95 % bootstrap interval. The three controls open the main block; the methods that use hand-written skills are below the line.*

Method	1M mean ± s.e.	1M IQM [95 % CI]	1B mean ± s.e.
PPO	2.17 ± 0.03	2.17 [2.11, 2.24]	11.72 ± 0.05
PPO-RNN	2.19 ± 0.03	2.19 [2.12, 2.26]	16.22 ± 0.27
PQN-RNN	1.36 ± 0.16	1.47 [1.03, 1.72]	16.10 ± 0.09
Count bonus (PPO-RNN)	2.27 ± 0.04	2.28 [2.18, 2.37]	16.33 ± 0.35
Count bonus (PQN-RNN)	1.48 ± 0.12	1.56 [1.26, 1.73]	15.99 ± 0.01
SIL	2.14 ± 0.01	2.14 [2.10, 2.17]	15.13 ± 0.06
Combination	2.13 ± 0.03	2.12 [2.07, 2.19]	15.35 ± 0.32
Uses hand-written skills			
SMDP skills	0.83 ± 0.21	0.91 [0.39, 1.35]	7.98 ± 0.37
Dyna-Q	1.08 ± 0.20	1.20 [0.66, 1.54]	8.76 ± 0.67

Table 2: *Reproduction of the controls: our mean score (%) against the published one, the gap and the acceptance band h (the check is two-sided).*

Control	Budget	Ours	Published	Gap	h
PPO	1M	2.17	2.20	-0.03	0.13
PPO-RNN	1M	2.19	2.30	-0.11	0.14
PPO	1B	11.72	11.90	-0.18	0.63
PPO-RNN	1B	16.22	15.30	0.92	0.63
PQN-RNN	1B	16.10	16.00	0.10	0.90

Table 3: *Ablations at 1M (10 seeds): mean ± s.e. and IQM of each ablation, each below its tuned parent.*

Variant	mean ± s.e.	IQM
Count bonus, SimHash key	1.78 ± 0.04	1.78
SIL, single buffer	2.14 ± 0.03	2.13
SMDP, random top level	0.27 ± 0.13	0.34
SMDP, Monte Carlo	0.48 ± 0.15	0.52
SMDP, no low-stat flags	0.45 ± 0.17	0.52
Dyna-Q, $\kappa = 0$	0.69 ± 0.19	0.68

5. Results

Table 1 gives the scores.

At 1M. The three PPO-family agents are indistinguishable: PPO 2.17, PPO-RNN 2.19, SIL 2.14, the combination 2.13. The progress bonus on PPO-RNN is the highest (2.27) but not significantly above its control (PoI 0.68, Holm $p = 1.000$). The PQN-RNN family is clearly lower (PQN-RNN 1.36, PoI against PPO 0.00, Holm $p = 0.002$ for its count-bonus variant), and the skill methods are lowest (Dyna-Q 1.08, SMDP 0.83; Holm $p = 0.001$ and 0.001 against PPO-RNN) but not significantly below their own control PQN-RNN. The greedy evaluation favours the recurrent PPO (PPO-RNN 1.56, PPO 0.91). No final evaluation episode reaches an iron pickaxe.

At 1B. Every agent scores several times higher, and the order changes (Fig. 2). PQN-RNN is now level with PPO-RNN (16.10 against 16.22), PPO is far behind (11.72), and the skill methods stay lowest (Dyna-Q 8.76, SMDP 7.98). Over the nine methods the 1M and 1B means correlate with Spearman 0.67 (95 % CI 0.28 to 0.72). The 1M advantages do not carry over: the bonus on PQN-RNN gains 9.20 % over its control at 1M and -0.74 % at 1B, SIL goes from -2.32 % to -6.75 % (change -4.43, the only change whose interval excludes zero). With three seeds a gap below about one point is unresolved, not equal, Table 2 compares the controls with their published scores: all agree within the acceptance band except PPO-RNN at 1B, which lies 0.92 above its published 15.30 % (a gap we did not trace). PPO, PPO-RNN and PQN-RNN stop improving in the last fifth of the budget on all three seeds; SIL and the combination do on none. Learning speed differs too: the share of the budget at which a method first reaches 90 % of its final score moves for most methods between the budgets (Fig. 3). Table 3 gives the 1M ablations: the SimHash variant of the progress bonus (1.78) and every ablation of the skill methods score below their tuned parent, while the buffer layout of SIL changes nothing (2.14 against 2.14). At 1B the SimHash variant aborts after a few million steps because its hash table overflows; the other 1B ablations are still being completed.

6. Interpretation

Distilling every final policy into decision trees (16 to 4096 leaves) shows that no tree matches any network on 90 % of states. At 256 leaves the trees agree more with the 1B networks than with the 1M ones (PPO-RNN 0.505 at 1M, 0.639 at 1B), yet the trees keep only a small part of the network's score when they act (closed-loop ratio of PPO-RNN 0.102 at 1B, against 0.335 at 1M): agreement on logged states is not enough to read these policies. The features the trees use also change with the budget. At 1M the previous action and the distances to water and trees lead; at 1B the facing direction leads for every method without skills, followed by whether the agent is asleep and whether the next tile is walkable, so a 16-leaf tree reads as "walk the way you face and act when blocked". The skill methods lean on the previous action and on the block they face. Removing the memory features changes the 256-leaf fidelity little for the recurrent methods.

7. Discussion and conclusion

The simple conclusion of 1M, that a progress bonus helps a recurrent PPO, is not supported at either budget: the gain is inside the noise at 1M and gone at 1B. Hand-written skills lower the score at both budgets in our setting, clearly so at 1B. Limits: three seeds at 1B, no tuning at 1B, one environment version per budget, two GPU classes at 1B, and the PPO-RNN reproduc-



Figure 1: *Training score (% of the maximum) over environment steps at 1M (left) and 1B (right); controls dashed, bands are 95 % bootstrap intervals over seeds. The 1B curves keep rising for several methods.*



Figure 2: *Left: IQM and mean with 95 % intervals at 1M. Right: gain over the base control at 1M against 1B, in percent; on the diagonal the advantage is unchanged, below it the method loses ground at 1B.*



Figure 3: *Left: mean score of every method at 1M against 1B (log axes). Right: share of the budget at which the smoothed training score first reaches 90 % of its final value, 1M against 1B; below the diagonal the method gets there earlier at 1B.*

tion gap. The 1M and 1B runs used different software stacks; the cluster rerun of the controls shows differences within seed noise.

Reproducibility. Every number and table here is generated from the MLflow store; each run stores its configuration, package versions and commit, and a 1B run takes a few hours on one GPU.

Future work. Tune the methods at 1B, run more 1B seeds so close methods can be ordered, replace the hand-chosen progress summary by a learned one, and add the intrinsic-reward baselines of the Craftax paper on the same protocol.

Release. Code, results and the demo: `https://github.com/Pawlo77/rl-craftmax`, `https://huggingface.co/datasets/Pawlo77/wut-murl-craftax-results` and `https://huggingface.co/spaces/Pawlo77/wut-murl-craftax`; the project videos are linked there.

8. Generative AI Use Disclosure

The text of this report, the analysis code, the figures and the videos' narration were written with Claude Code (Anthropic) under the author's direction; the author chose the questions, the protocol and the methods, ran the experiments and takes responsibility for the content. The video narration is synthetic speech.

9. Acknowledgements

The 1B experiments ran on the EDEN GPU cluster (A100 and H100). Craftax and the baselines are by Matthews et al. (MIT licence).

10. References

[1] M. Matthews, M. Beukman, B. Ellis, M. Samvelyan, M. Jackson, S. Coward, and J. Foerster, "Craftax: A lightning-fast benchmark for open-ended reinforcement learning," in *Proc. ICML*, 2024. [Online]. Available: https://arxiv.org/abs/2402.16801

[2] M. Gallici, M. Fellows, B. Ellis, B. Pou, I. Masmitja, J. N. Foerster, and M. Martin, "Simplifying deep temporal difference learning," in *Proc. ICLR*, 2025. [Online]. Available: https://arxiv.org/abs/2407.04811

[3] J. Oh, Y. Guo, S. Singh, and H. Lee, "Self-imitation learning," in *Proc. ICML*, 2018. [Online]. Available: https://arxiv.org/abs/1806.05635

[4] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[5] R. Agarwal, M. Schwarzer, P. S. Castro, A. C. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in *Advances in Neural Information Processing Systems*, 2021. [Online]. Available: https://arxiv.org/abs/2108.13264

[6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017. [Online]. Available: https://arxiv.org/abs/1707.06347